

SUSHANT LOKHANDE

San Jose, CA | +1 669-340-6655 | lokhandesushant014@gmail.com | sushantlokhande.me |
linkedin.com/in/sushantlokhande14 | github.com/sushantlokhande14

EXPERIENCE

San Jose State University | Graduate Research Assistant, Distributed Real-Time Systems Aug 2025 to May 2026

- Reduced rebuffering by **19%** versus a fixed-rate baseline by building a **Docker**, **WebTransport**, and **Media-over-QUIC** testbed and running controlled adaptation experiments across network and protocol configurations.
- Cut experiment setup effort by about **55%** by automating orchestration, configuration, **telemetry** collection, and analysis, making large experiment sweeps reproducible.
- Built and exercised **14 interoperability scenarios** covering networking, dependency, and **failure recovery** behavior in a distributed real-time system.

Turito | Software Engineering Intern, Real-Time Systems Jun 2022 to Oct 2022

- Slashed p95 CPU by **22%** by profiling high-frequency media paths and removing excess buffer allocation, payload copies, repeated routing lookups, and **lock contention**.
- Cut controlled-disruption failures by **42%** (3.1% to 1.8%) by hardening **retries**, timeouts, failure recovery, and distributed state transitions.
- Prototyped a **Rust** + **QUIC** media path that supported about **8,000** lightweight subscribers with about **24%** lower startup time.
- Worked across a **WebRTC** control plane and **RTP/RTCP** media paths spanning about **12 regions**, including ICE/STUN/TURN connection setup and degraded-network handling.
- Reduced production diagnosis time by **35%** by adding structured **metrics**, runtime-state visibility, and targeted logs.

Trivia Softwares | Software Engineering Intern Nov 2021 to Jan 2022

- Built a **Rust/Tokio** load-testing framework driving about **25,000** concurrent sessions, exposing lock contention, serialized **WebSocket** writes, and slow session cleanup.
- Surfaced backend fixes that raised equivalent-load signaling throughput by about **31%**.
- Reduced performance-investigation time by about **40%** with tooling built on Rust workers, **Python** aggregation, **SQLite**, Flask, and Chart.js.

TECHNICAL SKILLS

Languages: Rust, Python, C++17, TypeScript, SQL

Distributed & Real-Time: Tokio, WebSockets, WebRTC, RTP/RTCP, QUIC, gRPC, Kafka, Temporal

Data & Infra: Redis, PostgreSQL, SQLite, Docker, AWS, Linux

Performance: profiling, lock contention analysis, load testing, AVX2/SIMD, memory layout

PROJECTS

Proxima, Vector Search Engine | C++17, AVX2/SIMD, HNSW, pybind11

- Built an HNSW vector search engine from scratch that reaches **0.999 recall** at **12,049 QPS** single-threaded, **1.8x** faster than hnswlib and **2.5x** faster than FAISS on 100K vectors.
- Cut traversal memory by **75%** (51 MB to 12.8 MB) with **int8** traversal on AVX2 kernels plus exact float re-ranking, and cache-aware node reordering.

Portcullis, Policy Gateway | Rust

- Built a **Rust** proxy that centralizes authorization and rate limiting before requests reach services and emits auditable accept, reject, and throttle decisions.

OPEN SOURCE

- fission/fission:** 3 merged PRs, Kubernetes event and KEDA object capture in support dumps, removed duplicate package enumeration
- corsairdev/corsair:** 6 merged **webhook signature verification** fixes (Telegram, Teams, SharePoint, Monday, GitLab, Jira) that fail closed

EDUCATION

San Jose State University | M.S. in Computer Science | GPA 3.8/4.0 | Spartan Scholar Aug 2024 to May 2026

Coursework: Distributed Computing, Advanced Algorithms, Database Systems, Computer Networks, GPU Programming, Machine Learning

University of Mumbai | B.Tech. in Computer Engineering | GPA 3.9/4.0 Jun 2019 to May 2023

Research: malware image-classification ensemble selected for Dr. Mark Stamp's ML textbook; sole-author contactless palmprint verification (0.9975 AUC)